SMART EXPENSE ANOMALY DETECTOR

Software Specification Document (SSD) | Version 2.0 | 29 October 2026 | Draft for review

Purpose and intended outcome

Build a multi-page web application where users securely sign in, record personal expenses, review spending summaries, and receive clear explanations for unusual transactions. React provides the UI, FastAPI validates requests and applies deterministic anomaly rules, and PostgreSQL persists data.

Scope update in this version

This revision replaces the earlier single-demo-user assumption. It includes authentication, protected pages, user-specific expense isolation and logout. Demo transactions shall not be presented as real user data.

Technology stack

Layer	Technology	Responsibility
Frontend	React + JavaScript + React Router	Login, Dashboard, Add Expense, Transactions and navigation.
Services	Axios; separate service modules	Central API setup and dedicated auth, expense and dashboard calls.
Backend	FastAPI + Python	Authentication, authorization, validation, expense endpoints and anomaly detection.
Database	PostgreSQL	Users, categories, expenses, flags and explanations.

Core user journey

Login → dashboard → add expense → FastAPI detects anomalies and saves → review transactions → logout. Users may access only their own expenses.

Document guide

Page	Coverage
1	Purpose and stack
2	Scope, actors and access control
3	Functional requirements
4	Anomaly detection
5	UI, navigation and service separation
6	Data model and API contract
7	Security and acceptance tests
8	Delivery plan and decisions

1. Scope, actors and access control

Included in the initial release

Users authenticate before opening protected pages. Each user can create and view their own expenses, inspect dashboard totals and category summaries, and review anomaly explanations. Logout ends the authenticated session. FastAPI enforces access independently of React.

ID	Class	Requirement
R01	Core	User must authenticate before accessing dashboard, add-expense or transaction pages.
R02	Core	User can create and retrieve only their own expenses.
R03	Core	Logout ends the session and redirects to login.
R04	Core	Unusual spending is flagged using explainable rules.
R05	Core	Every flag has a human-readable reason.
R06	Core	FastAPI validates required fields and permissions.
P01	Proposed	Use email/password authentication; store a secure password hash only.
P02	Proposed	React Router protects authenticated routes.
P03	Proposed	Backend derives identity from validated session/token; never trust client-supplied user IDs.

Actors

Registered user: logs in, records expenses, views their dashboard and transactions, and logs out. **Unauthenticated visitor:** can access login and any explicitly enabled registration page only. No administrator role is required.

Access matrix

Action	Visitor	Authenticated user
Open login	Allowed	Allowed
View dashboard	Redirect to login	Own data only
Add expense	Rejected	Allowed for self
List/read expenses	Rejected	Own data only
Read another user's expense	Rejected	Forbidden or not found
Logout	Redirect to login	End session and clear client state

Out of scope

Bank integration, receipt OCR, machine-learning models, recurring payment prediction, multi-currency conversion, shared accounts, exports and email/SMS alerts are excluded from the MVP.

2. Functional requirements and page behavior

ID	Requirement
FR01	Login page collects email/password, validates required fields and submits to FastAPI.
FR02	Valid login navigates to /dashboard. Invalid credentials show a generic error.
FR03	Protected routes redirect to /login when no valid authenticated session exists.
FR04	Dashboard shows total spending, transaction count, flagged count, category totals and recent saved expenses for the signed-in user.
FR05	Add Expense collects amount, category, date and description/merchant and submits to FastAPI.
FR06	After successful save, display saved result and API-returned anomaly reasons; refresh summaries.
FR07	Transactions lists only the current user's saved expenses and supports category/date filters.
FR08	Flagged transactions show reasons. A flag is advisory and does not block a valid expense.
FR09	Logout ends/revokes the session where supported, clears client auth state and navigates to /login.
FR10	No demo expenses appear as real data. Empty accounts show a useful empty state.
FR11	Failed requests show an error and never imply data was saved.
FR12	Backend validation and authorization remain authoritative.

Primary navigation

Authenticated navigation includes Dashboard, Add Expense, Transactions and Logout. Login does not display authenticated navigation. Revisiting protected URLs after logout must redirect to login.

Empty and error states

New users see zero totals and “No expenses found” rather than fabricated transactions. Show loading indicators while requests are pending. Validation errors identify fields where possible. Never expose stack traces, SQL statements or secrets.

3. Anomaly detection and explanations

The MVP uses deterministic rules in backend module anomaly_detector.py. FastAPI evaluates the authenticated user's history, saves the expense and rule results, and returns the explanation to React.

Rule	Condition	Explanation
AN01 — High amount	Amount ≥ ■1,500 (default threshold).	“This expense is high-value because it is at least ■1,500.”
AN02 — Above category norm	At least 3 prior expenses in category; new amount >2× average of previous 10 category expenses (or all available if fewer).	“This is {x}× your recent average for {category}.”
AN03 — Large jump	At least 3 prior expenses; amount > prior category maximum by ≥■500 and >1.5× that maximum.	“This is much higher than your previous {category} expenses.”

Rule evaluation requirements

Historical comparisons use only the authenticated user's prior expenses in the same category and exclude the new transaction. AN02 and AN03 do not run with fewer than 3 prior expenses. AN01 can run without history.

API response

Return is_flagged (boolean), anomaly_reasons (array of strings) and rule_codes (array of stable identifiers). is_flagged is true when one or more rules trigger.

Examples

Scenario	Expected result
Coffee ■2,000	Saved and flagged by AN01; explanation names the ■1,500 threshold.
Groceries ■3,000; recent average ■900 and enough history	Flagged by AN02; explanation includes category and comparison.
■450 expense below threshold and within history	Not flagged unless another rule triggers.
■1,800 expense with no category history	Flagged by AN01 only; no claim it exceeds usual spending.

Important distinction

A flag means “review this expense,” not “fraud detected.” The app must not block a valid transaction just because it is unusual. If history is insufficient, state that there is not enough category history for a comparison.

4. UI, navigation and frontend architecture

Page/component	Required behavior
Login.jsx	Email/password form, validation, loading state, generic error and navigation after successful login.
Dashboard.jsx	Fetch summary via dashboardService.js; show totals, flagged count and category totals.
AddExpense.jsx	Controlled form; call expenseService.js; show API-returned reasons after confirmed save.
Transactions.jsx	Fetch/filter persisted expenses; show date, description, category, amount, status and reasons.
Navbar.jsx / Sidebar.jsx	Navigation links and logout action for authenticated pages.
ProtectedRoute.jsx	Redirect to /login when authenticated state is absent or invalid.

Frontend service separation

File	Responsibility
services/api.js	Shared Axios instance, base URL and common request configuration.
services/authService.js	Login, logout and current-user/session requests.
services/expenseService.js	Create expense, list expenses and filters.
services/dashboardService.js	Fetch dashboard totals and category summaries.

Navigation and behavior

Routes: /login, /dashboard, /add-expense and /transactions. Login is the initial route for unauthenticated users. Authenticated users navigate between protected pages until logout or session expiry.

Architecture requirements

React components shall not contain raw endpoint logic; they call the relevant service module. Configure the API base URL centrally. Frontend validation improves usability, but FastAPI remains authoritative.

Accessibility and feedback

Support keyboard use, visible focus and associated labels. Communicate flags with text, not color alone. Disable submit while pending, preserve form values after failures, and show success only after the API confirms persistence.

5. Data model and REST API contract

Proposed PostgreSQL tables

Table	Fields and constraints
users	id UUID primary key; email UNIQUE NOT NULL; password_hash NOT NULL; created_at TIMESTAMPTZ. Never store plain-text passwords.
categories	id UUID/serial primary key; name UNIQUE NOT NULL. Seed Food, Coffee, Transport, Shopping, Bills, Health, Entertainment, Travel and Other.
expenses	id UUID primary key; user_id FK NOT NULL; amount NUMERIC(12,2) CHECK >0; category_id FK; description VARCHAR(120); expense_date DATE; is_flagged BOOLEAN; anomaly_reasons JSONB; created_at TIMESTAMPTZ.

Validation rules

Amount must be greater than zero and at most ■10,00,000. Description is required, 2–120 trimmed characters. Category must exist. Expense date is required and cannot be more than one day in the future. Ownership IDs are server-controlled.

REST API — /api/v1

Method/path	Contract
POST /auth/login	Validate credentials; return session/token or controlled authentication error.
POST /auth/logout	End/revoke session or token where supported.
GET /auth/me	Return safe profile fields for current authenticated user.
GET /categories	Return available categories.
POST /expenses	Validate, detect anomalies and create expense for authenticated user; return 201 with reasons.
GET /expenses	Return current user's expenses only; support category, start_date and end_date filters.
GET /dashboard/summary	Return total_spending, expense_count, flagged_count and category_totals for current user/date range.

HTTP errors

Status	Meaning
401	Missing, expired or invalid authentication.
403 / 404	Forbidden action or concealed inaccessible resource.
422	Invalid fields, category, amount or date.
500 / 503	Controlled unexpected error or temporary outage; no internal details.

6. Security, quality and acceptance tests

Security and operational requirements

ID	Requirement
NFR01 Authentication	Hash passwords using an appropriate algorithm. Login errors do not disclose whether an email exists.
NFR02 Authorization	FastAPI validates identity on protected requests and scopes all expense reads/writes to authenticated user.
NFR03 Session safety	Document session/token strategy, expiry and logout. Client route protection alone is not security.
NFR04 Secrets	Keep DB credentials and signing secrets in backend environment configuration; never expose in React.
NFR05 Database safety	Use parameterized queries/ORM and foreign keys. Saved expenses survive restart.
NFR06 Consistency	Dashboard summaries and transaction rows reflect the same persisted records and filters.
NFR07 Maintainability	Anomaly rules live in anomaly_detector.py; frontend requests live in separate service modules.
NFR08 Usability	Responsive pages include loading, empty, success and error states.

Acceptance tests

Test	Action	Expected result
TC01	Open /dashboard without session.	Redirect to /login; no protected data.
TC02	Submit valid credentials.	Authenticated state established; navigate to dashboard.
TC03	Submit invalid credentials.	Generic error; no access.
TC04	Logout, then revisit /transactions.	Redirect to login; protected content inaccessible.
TC05	Create valid expense.	201; one expense persists for current user.
TC06	Create ■2,000 coffee expense.	Saved and flagged by AN01 with threshold explanation.
TC07	Submit negative amount/missing category.	422; no record created.
TC08	Try to read another user's expense.	Rejected/concealed; no data exposure.
TC09	Request dashboard summary.	Totals match user's persisted expenses.
TC10	Refresh after successful save.	Expense and explanation remain visible.
TC11	Simulate API/database outage.	No false success; form values retained.
TC12	Use keyboard/narrow viewport.	Labels, focus, navigation and actions usable.

7. Delivery plan and decisions

Suggested implementation sequence

Phase	Work	Deliverable
1	Initialize React/Vite, FastAPI, PostgreSQL and environment variables.	Project skeleton and DB connection.
2	Implement user model, password hashing, login/logout/current-user endpoints and auth dependencies.	Backend authentication.
3	Create React Router pages, ProtectedRoute and separate service modules.	Multi-page navigation and service separation.
4	Create category/expense endpoints, validation and ownership checks.	Persisted create/list flow.
5	Implement anomaly_detector.py and dashboard summary endpoint.	Explainable flags and totals.
6	Connect pages to services and remove demo data; add empty/error/loading states.	End-to-end connected UI.
7	Run acceptance tests for authentication, isolation, anomaly rules and logout.	Demo-ready MVP.

Decisions requiring review

Decision	Proposed baseline	Effect if changed
Account creation	Enable simple registration or provision accounts manually.	Registration adds endpoint and UI.
Authentication transport	Choose secure session or token design before coding.	Affects Axios, storage, expiry and logout.
High-value threshold	■1,500 for AN01.	Changes which transactions are flagged.
Personal baseline	Previous 10 category expenses; minimum 3.	Changes comparisons.
Categories	Fixed seeded list.	Custom categories add UI/API scope.
Editing/deleting	Excluded from MVP.	Requires revalidation and re-detection.

Definition of ready

Ready for demo when login/logout work with FastAPI, protected routes are enforced by the API, users cannot access each other's expenses, data persists in PostgreSQL, anomaly reasons are generated by the backend, dashboard totals match saved data, and acceptance tests have been checked.

Change control

Update this SSD, API contract, data model and affected tests before changing requirements. This document describes intended behavior; it does not claim implementation or testing is complete.

End of specification • Version 2.0 • 8 pages